NoSQL Databases

Francesco Pelacani
2025-2026

Key-Value Databases

What is a Key-Value store?

Is a simple **hash table memory-first key-value data store**, used when all access to the database is via primary key.

Features

- **Consistency** → Only for operations on a single key
- **Transactions** → Generally there are no guarantees on the writes
- **Query features** → KV stores can query by the key, *and that's it*
- **Structure of data** → The Value can be a blob, text, JSON, …
- **Scaling** → Sharding by key

Usage of Key-Value Databases

Suitable Use Cases

1. Storing Session Information (shopping cart)
2. Cache user profiles and preferences
3. Search and Query
4. Vector search (recommendation, chat bots)

When Not to Use

1. Relationships among Data
2. Multi Operation Transaction
3. Query by Data
4. Operations by Sets

Redis

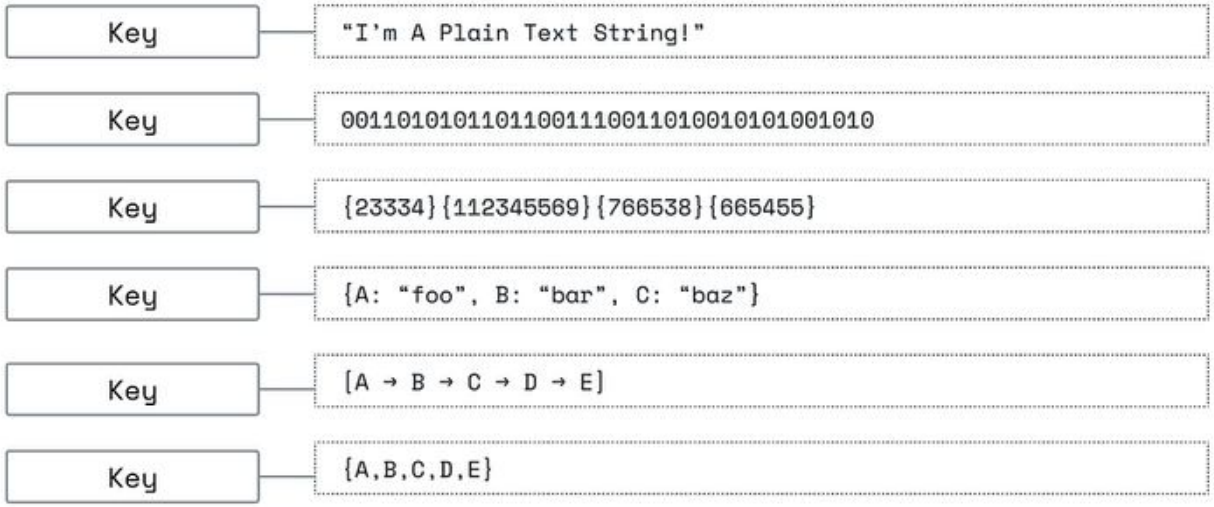
practice

Practice - Get started with Redis

Goto: link to Redis University

Ok, but what is a Key-Value then?

A **value** is a piece of data you store and a **key** is the name for that data; together they are called *key-value pair*.



Redis usages

Redis is known as i**n-memory data structure store**, with sub-millisecond read/write, widely used as key-value DB.

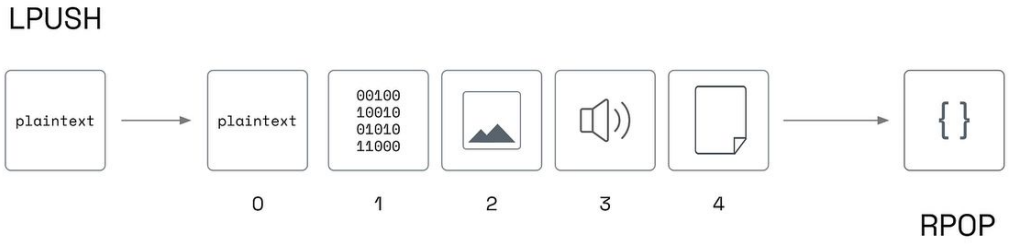
Its typical roles are cache, session store, rate-limiter, message queues / lightweight streaming.

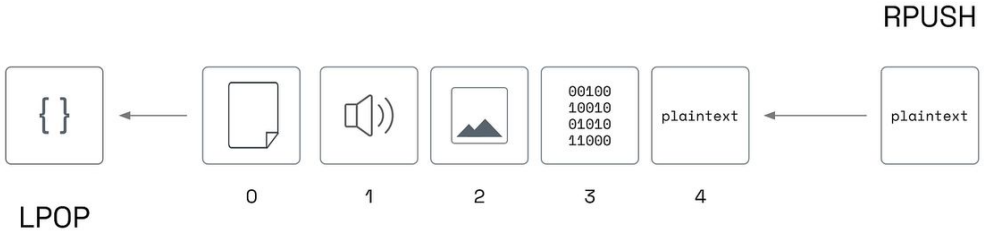
Naming keys

Use consistent naming convention to avoid confusion and collision.
In **Redis** you can use **Keyspaces**. They group related keys together using a hierarchical set of prefixes.

Lists

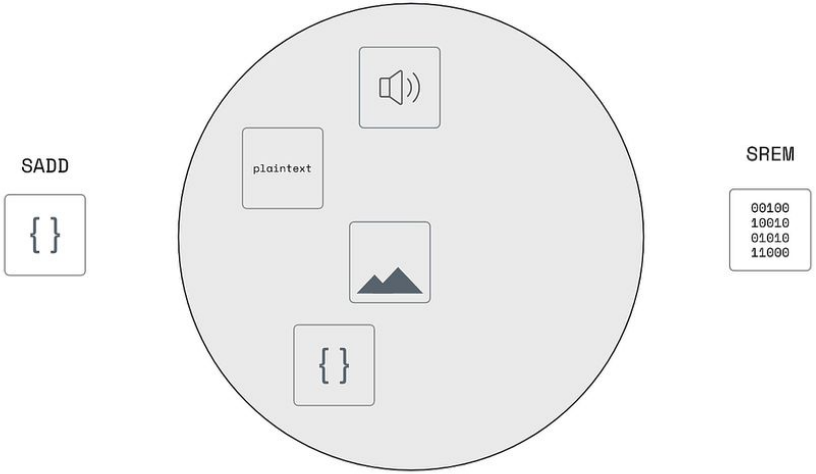
A list is an ordered group of elements.
Can be manipulated through **pushing** and **popping**.





Sets

Unordered collection of **unique members**.

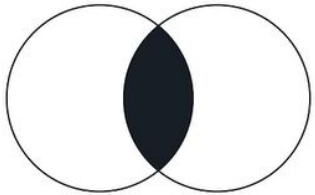


Sets operations

union

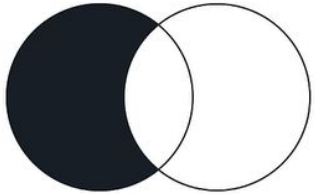
SUNION

intersection



SINTER

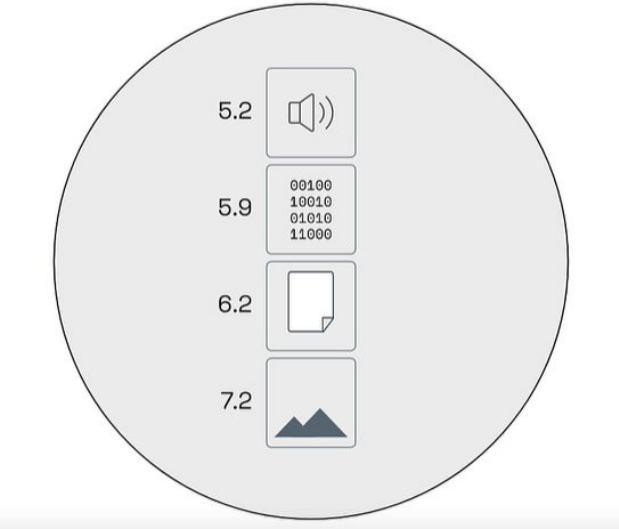
difference



SDIFF

Sorted Sets

They combine some of the feature of sets with some of the features of lists.
Each member is associated with a score.

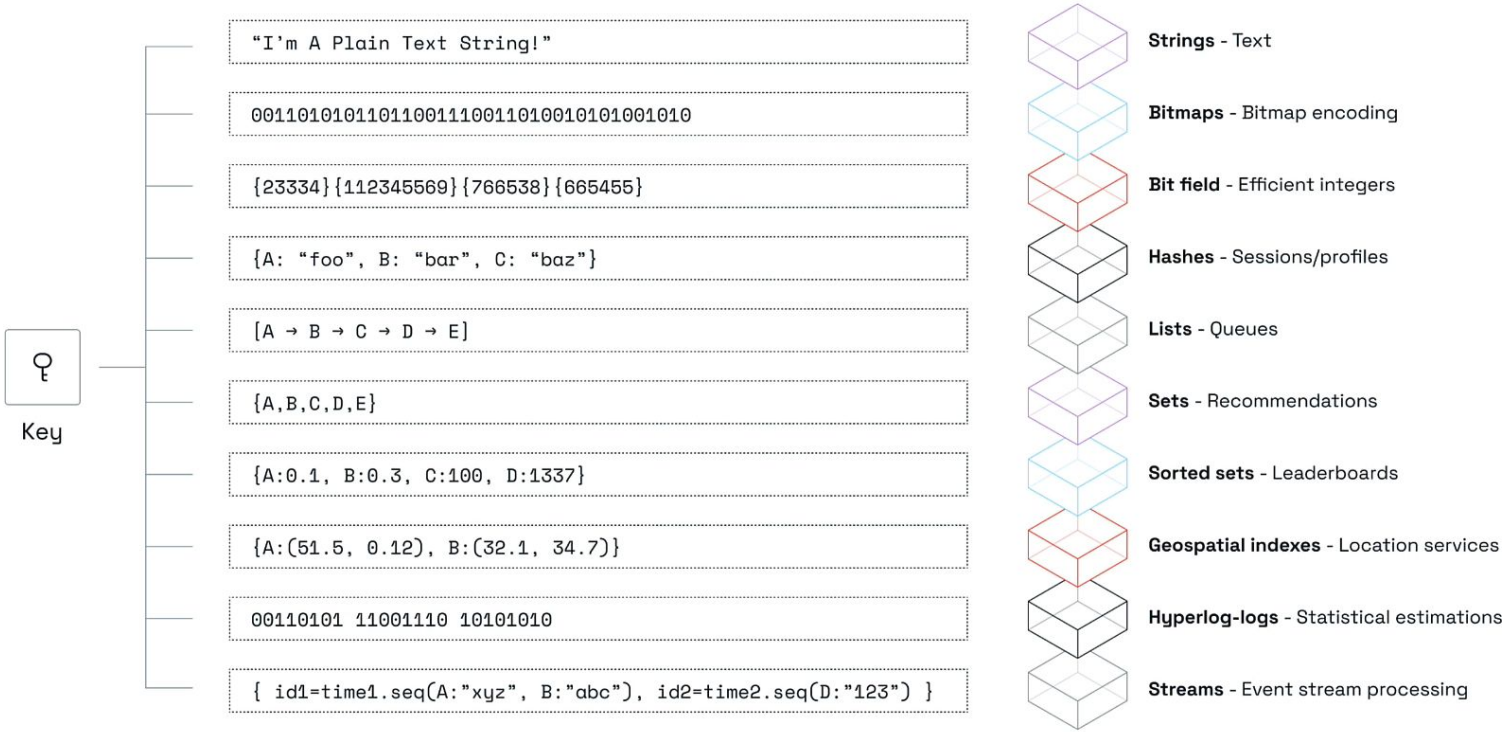


Hash

Is a collection of key-value pairs.
Redis itself is a giant map of keys and values.

Field	Value
color	red
profile-picture	

More Data Structures



Probabilistic Data Structures

Are types of structures that **sacrifice accuracy for improved performance**.
They give **approximations of statistics** such as counts, frequencies, and rankings rather than precise values.

The advantage of using approximations is that they are adequate for many common purposes but are much more efficient to calculate.

Key Expiration

Two types of keys:

- **Persistent** → default behaviour, stay around until explicitly removed
- **Volatile** → can have a TTL (Time-To-Live) or an expiration time, after which they are automatically removed

practice

Practice - Let's see a real example

Goto: GitHub